16A

CSS LAYOUT WITH FLEXBOX

About Flexbox

- **Flexbox** is a display mode that lays out elements along one axis (horizontal or vertical).

- Useful for menu options, galleries, product listings, etc.

- Items in a flexbox can expand, shrink, and/or wrap onto multiple lines, making it a great tool for responsive layouts.

- Items can be reordered, so they aren't tied to the source order.

- Flexbox can be used for individual components on a page or the whole page layout.

Flexbox Container

`display: flex`

- To turn on Flexbox mode, set the element's **`display`** to **`flex`**.

- This makes the element a **flexbox container**.

- All of its direct children become **flex items** in that container.

- By default, items line up in the writing direction of the document (left to right rows in left-to-right reading languages).

Without Flexbox

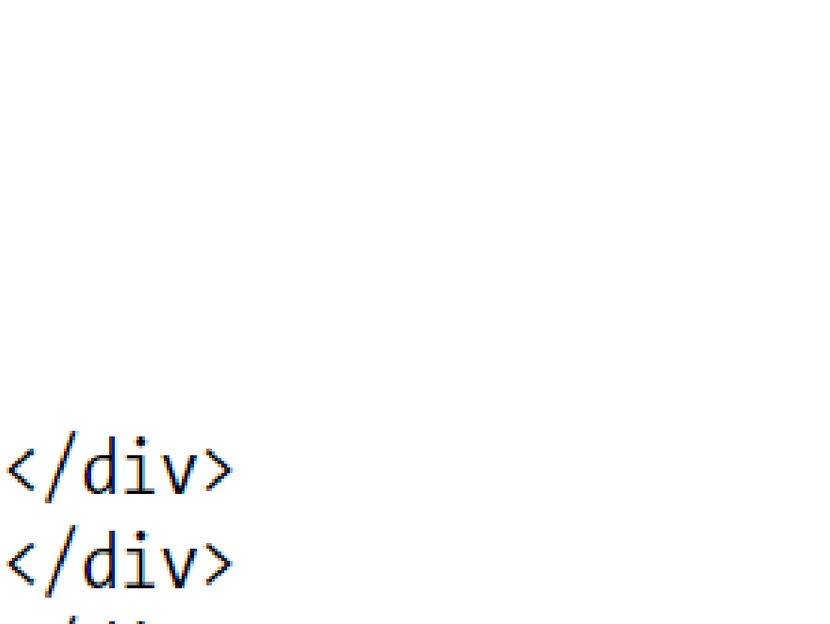
```
<head>
    <style>
        .box{ height: 20px; text-align: center;}
        .box1{ background-color: red;}
        .box2{ background-color: orange;}
        .box3{ background-color: green;}
        .box4{ background-color: blue;}
        .box5{ background-color: purple;}
    </style>
</head>
<body>
<div id="container">
    <div class="box box1">1</div>
    <div class="box box2">2</div>
    <div class="box box3">3</div>
    <div class="box box4">4</div>
    <div class="box box5">5</div>
</div>
</body>
```

THE MARKUP

```
<div id="container">
  <div class="box box1">1</div>
  <div class="box box2">2</div>
  <div class="box box3">3</div>
  <div class="box box4">4</div>
  <div class="box box5">5</div>
</div>
```

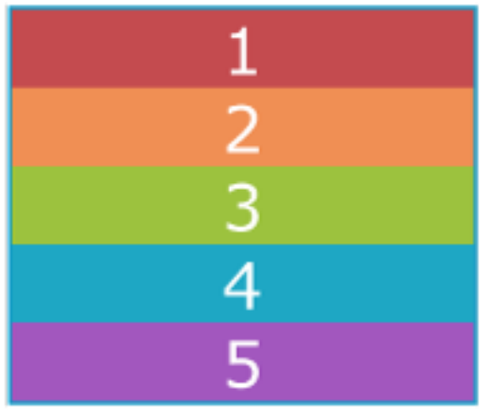
THE STYLES

```
#container {
  display: flex;
}
```



Flexbox Container (cont'd)

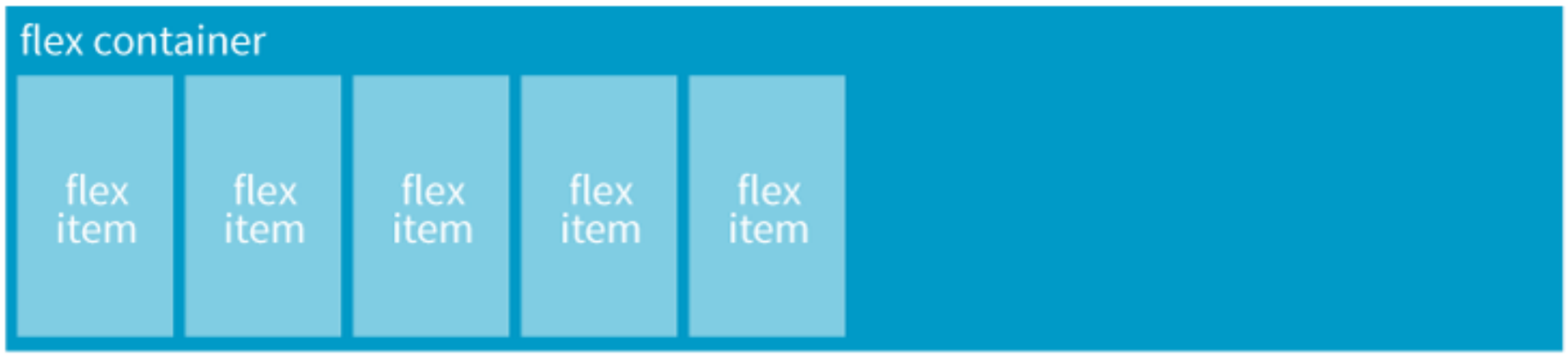
By default, the **divs** display as block elements, stacking up vertically. Turning on flexbox mode makes them line up in a row.



block layout mode

```
display: flex;
```

flexbox layout mode



Rows and Columns (Direction)

`flex-direction`

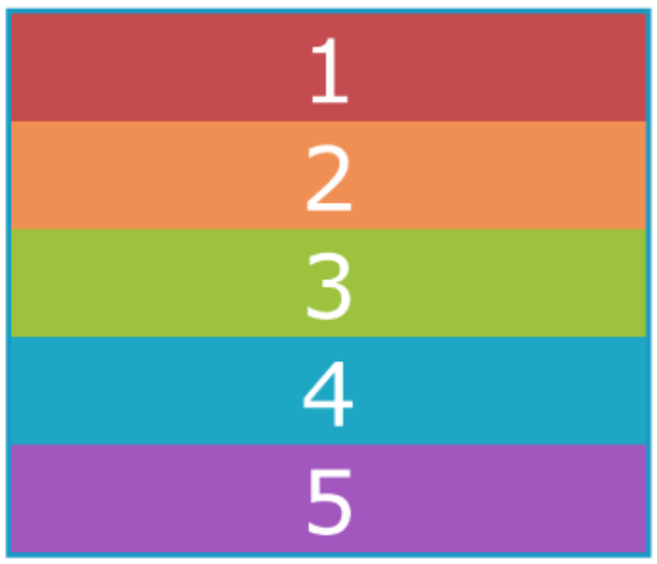
Values: `row`, `column`, `row-reverse`, `column-reverse`

The default value is **`row`** (for L-to-R languages), but you can change the direction so items flow in columns or in reverse order:

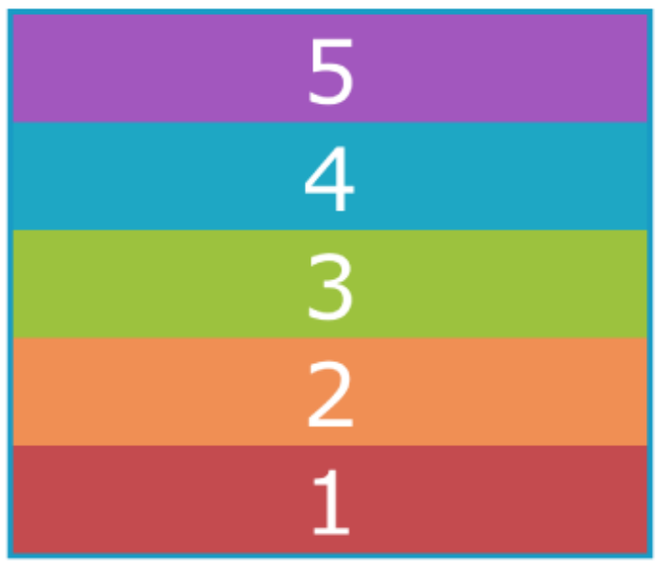
`flex-direction: row;` (default)

`flex-direction: row-reverse;`

`flex-direction: column;`



`flex-direction: column-reverse;`



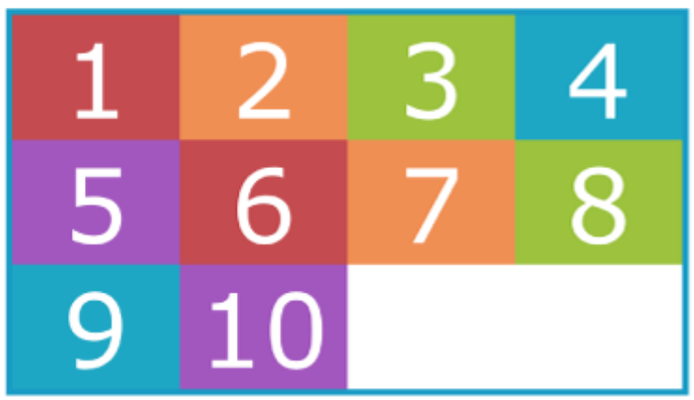
Wrapping Flex Lines

`flex-wrap`

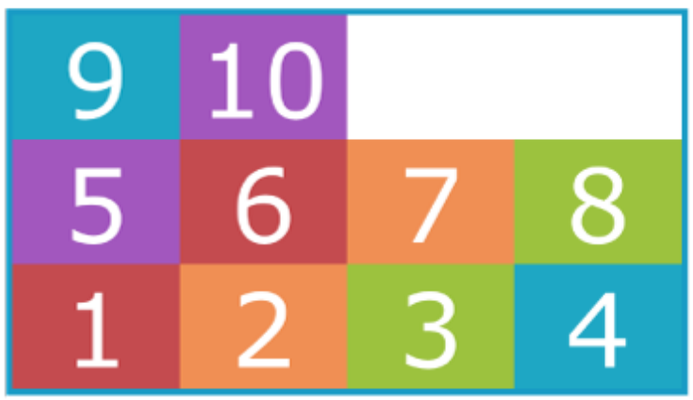
Values: `wrap`, `nowrap`, `wrap-reverse`

Flex items line up on one axis, but you can allow that axis to wrap onto multiple lines with the **`flex-wrap`** property:

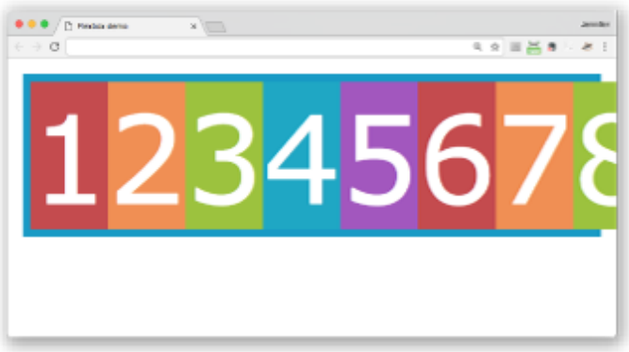
```
flex-wrap: wrap;
```




```
flex-wrap: wrap-reverse;
```



`flex-wrap: nowrap;` (default)



When wrapping is disabled, flex items squish if there is not enough room, and if they can't squish any further, may get cut off if there is not enough room in the viewport.

Flex Flow (Direction + Wrap)

`flex-flow`

Values: *Flex-direction flex-flow*

The shorthand **`flex-flow`** property specifies both direction and wrap in one declaration.

Example

```
#container {
  display: flex;
  height: 350px;
  flex-flow: column wrap;
}
```

Flexbox Alignment Terminology

- Flexbox is “direction-agnostic,” so we talk in terms of *main axis* and *cross axis* instead of rows and columns.

- The **main axis** runs in whatever direction the flow has been set.

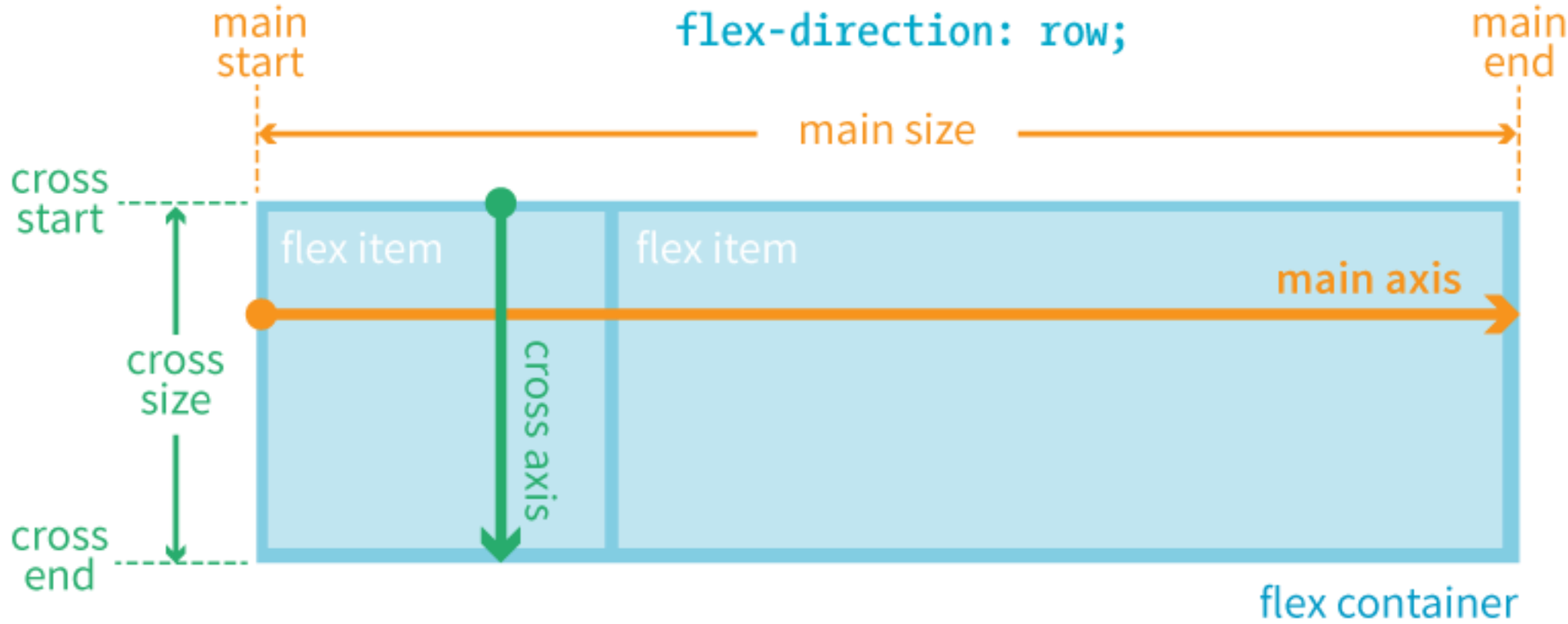
- The **cross axis** runs perpendicular to the main axis.

- Both axes have a **start**, **end**, and **size**.

ROW: Main and Cross Axes

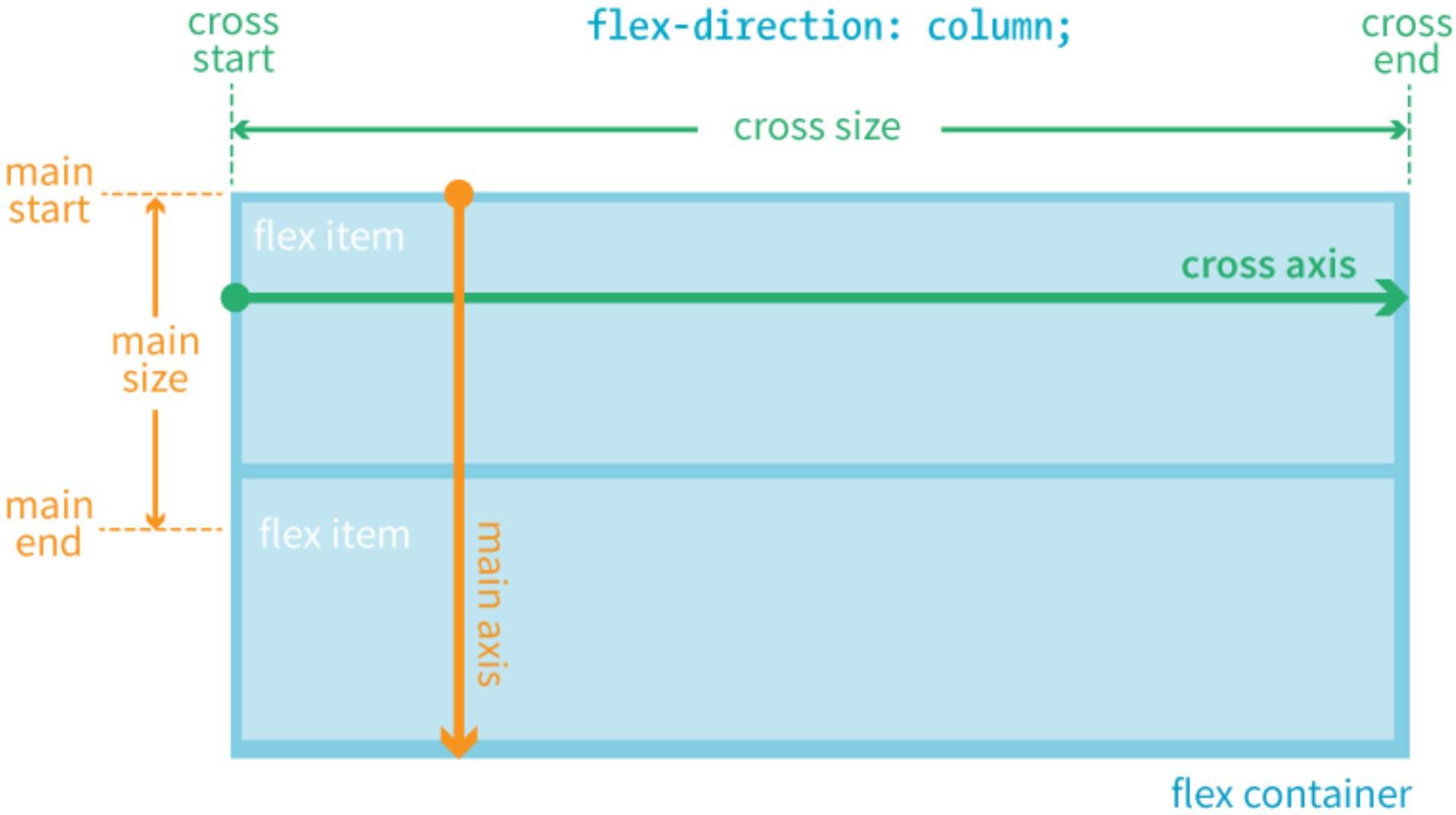
FOR LANGUAGES THAT READ HORIZONTALLY FROM LEFT TO RIGHT:

When `flex-direction` is set to `row`, the main axis is horizontal and the cross axis is vertical.



COLUMN: Main and Cross Axes

When **flex-direction** is set to **column**, the main axis is vertical and the cross axis is horizontal.



Aligning on the Main Axis

`justify-content`

Values: `flex-start`, `flex-end`, `center`, `space-between`, `space-around`

When there is space left over on the **main axis**, you can specify how the items align with the **`justify-content`** property (notice we say *start* and *end* instead of left/right or top/bottom).

The **`justify-content`** property applies to the **flex container**.

Example:

```
#container {
  display: flex;
  justify-content: flex-start;
}
```

Aligning on the Main Axis (cont'd)

When the direction is **row**, and the **main axis is horizontal**

`justify-content: flex-start;` (default)

`justify-content: flex-end;`

`justify-content: center;`

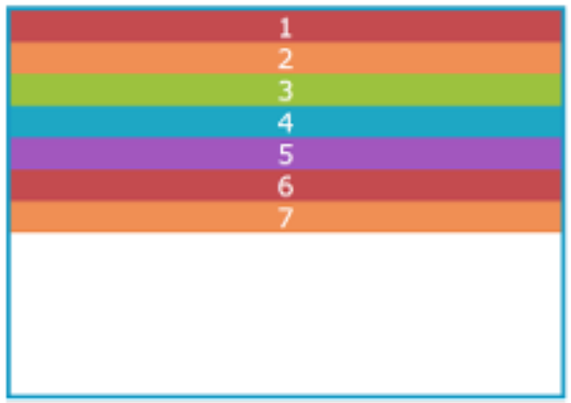
`justify-content: space-between;`

`justify-content: space-around;`

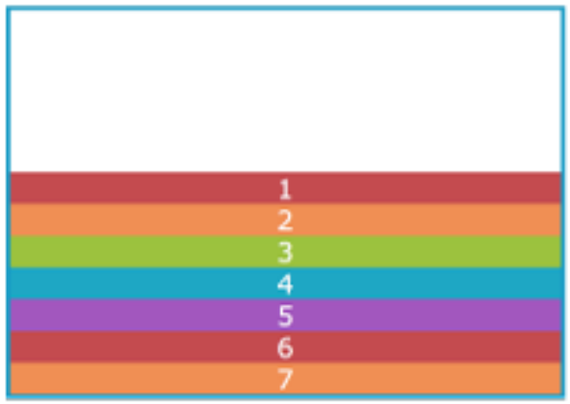
Aligning on the Main Axis (cont'd.)

When the direction is **column**, and the **main axis is vertical**

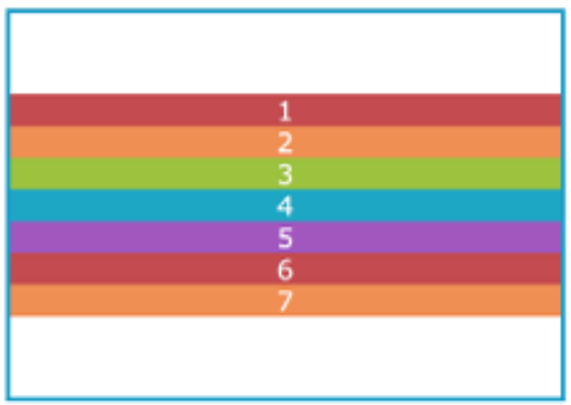
`justify-content: flex-start;` (default)



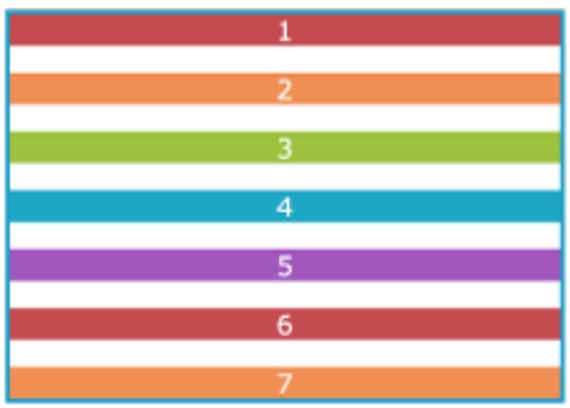
`justify-content: flex-end;`



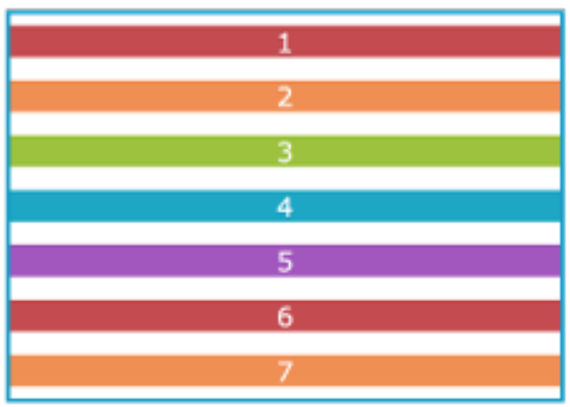
`justify-content: center;`



`justify-content: space-between;`



`justify-content: space-around;`



NOTE: I needed to specify a height on the container to create extra space on the main axis. By default, it's just high enough to contain the content.

Aligning on the Cross Axis

`align-items`

Values: `flex-start`, `flex-end`, `center`, `baseline`, `stretch`

When there is space left over on the **cross axis**, you can specify how the items align with the **`align-items`** property.

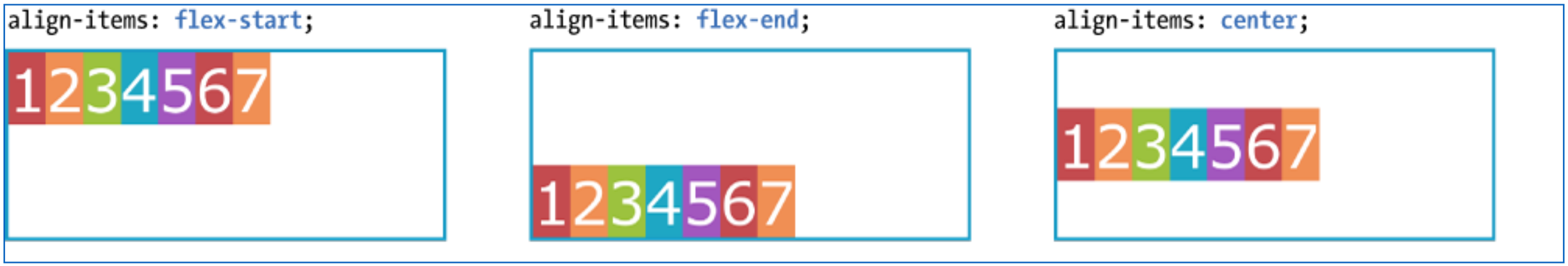
The **`align-items`** property applies to the **flex container**.

Example:

```
#container {
  display: flex;
  flex-direction: row;
  height: 200px;
  align-items: flex-start;
}
```

Aligning on the Cross Axis (cont'd)

When the direction is **row**, the main axis is horizontal, and the **cross axis is vertical**.



NOTE: I needed to specify a height on the container to create extra space on the cross axis. By default, it's just high enough to contain the content.

Aligning on the CROSS Axis (cont'd)

`align-self`

Values: `flex-start`, `flex-end`, `center`, `baseline`, `stretch`

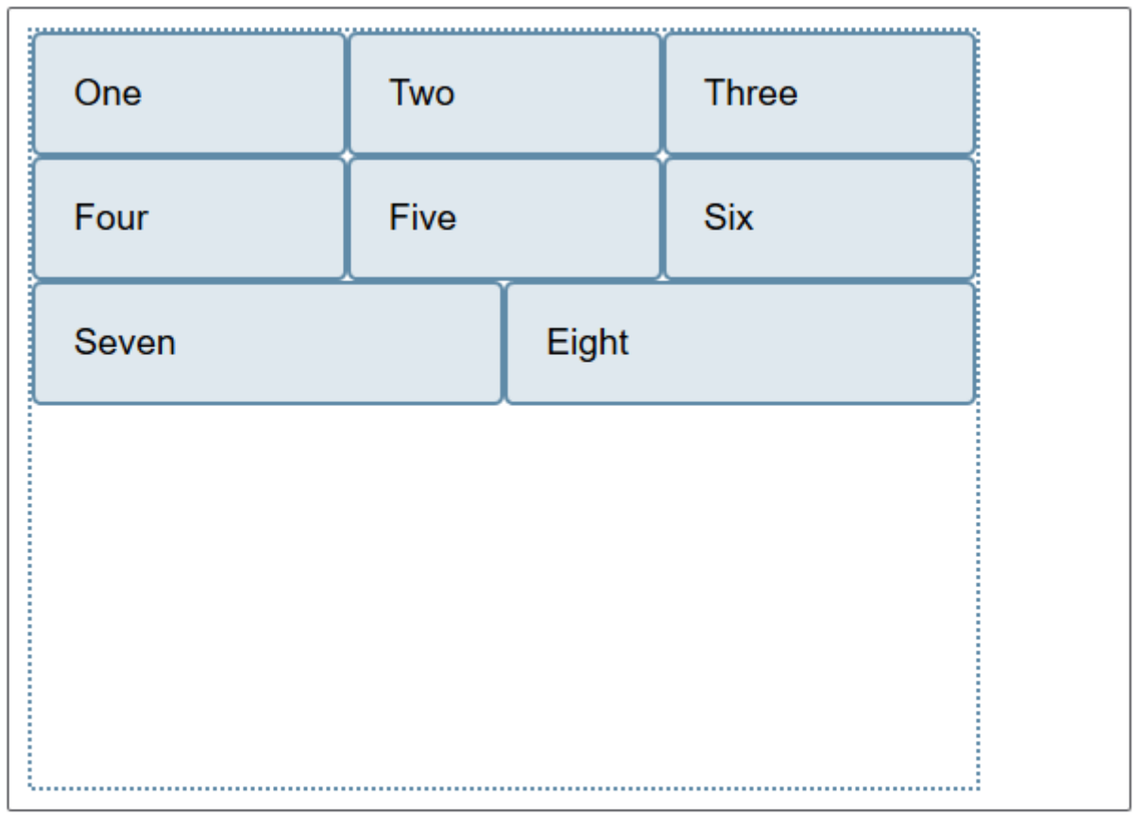
Aligns an **individual item** on the cross axis. This is useful if one or more items should override the **`align-items`** setting for the container.

The **`align-self`** property applies to the **flex item**.

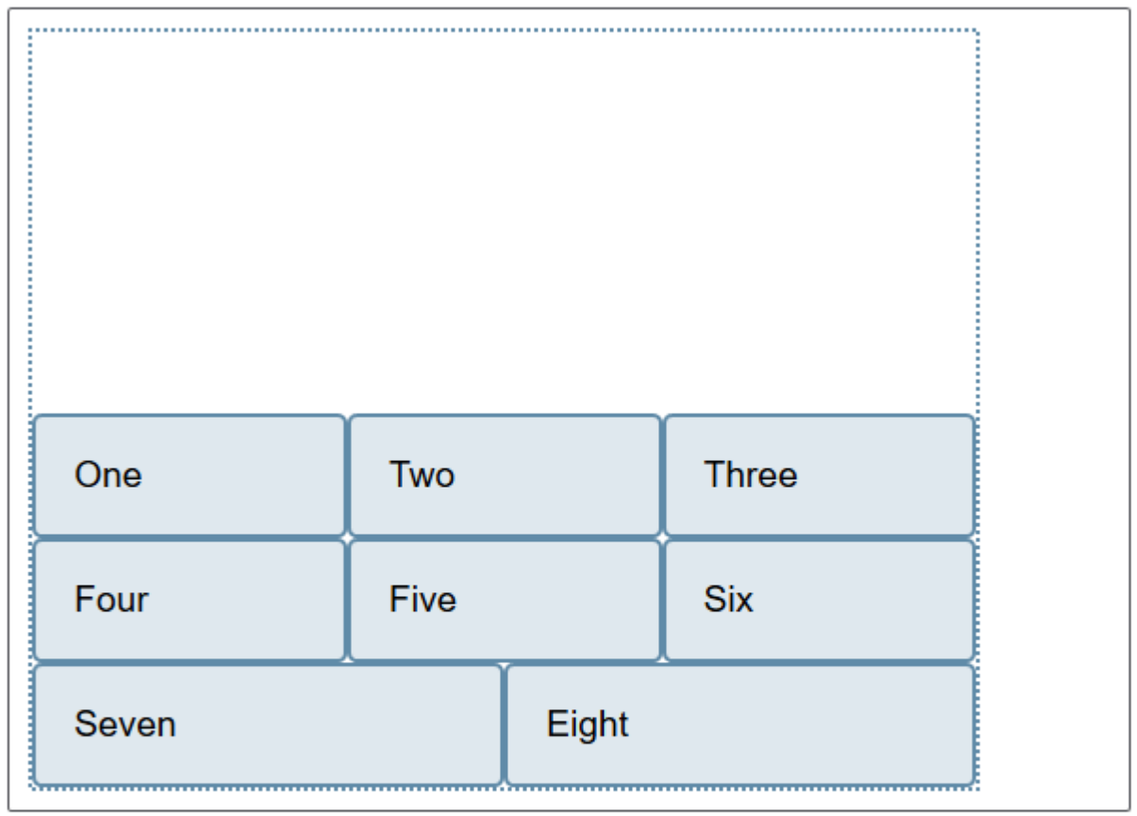
Example:

```
.box4 {
  align-self: flex-end;
}
```

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout/Aligning_Items_in_a_Flex_Container

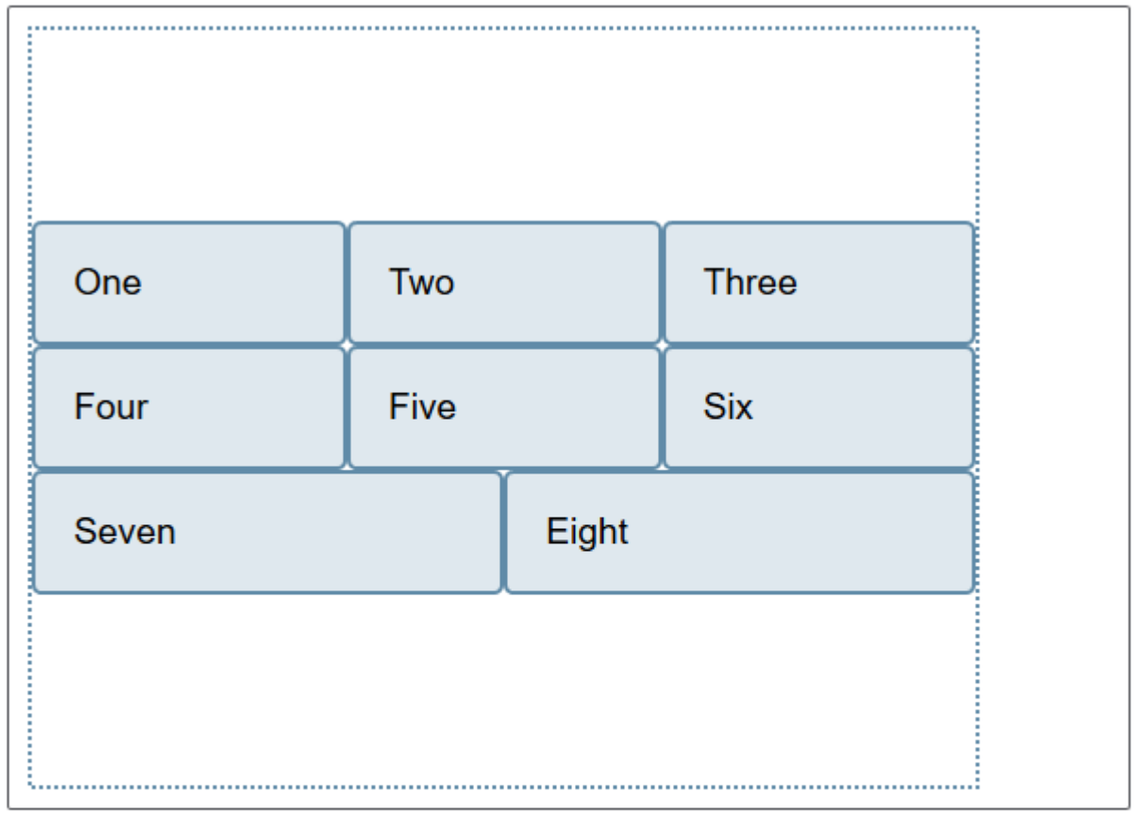



```
.box {
  display: flex;
  flex-wrap: wrap;
  height: 400px;
  align-content: flex-start;
}
```

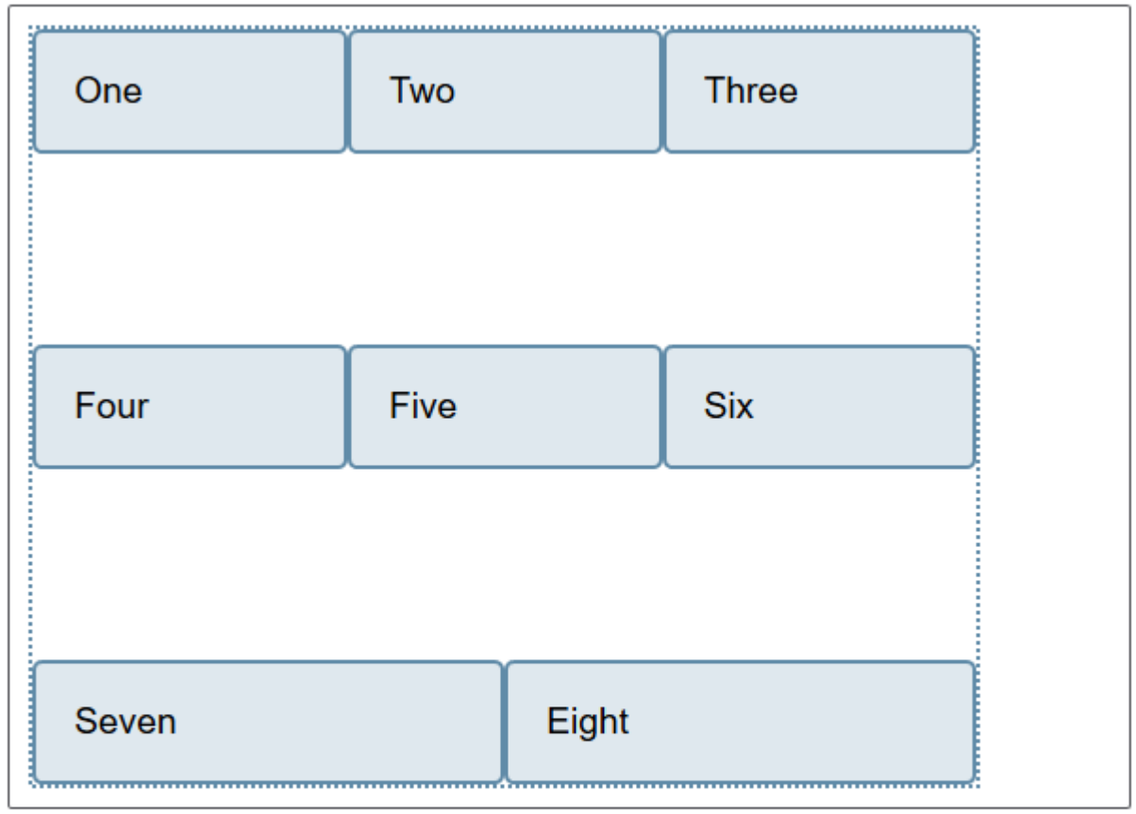



```
.box {
  display: flex;
  flex-wrap: wrap;
  height: 400px;
  align-content: flex-end;
}
```

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout/Aligning_Items_in_a_Flex_Container

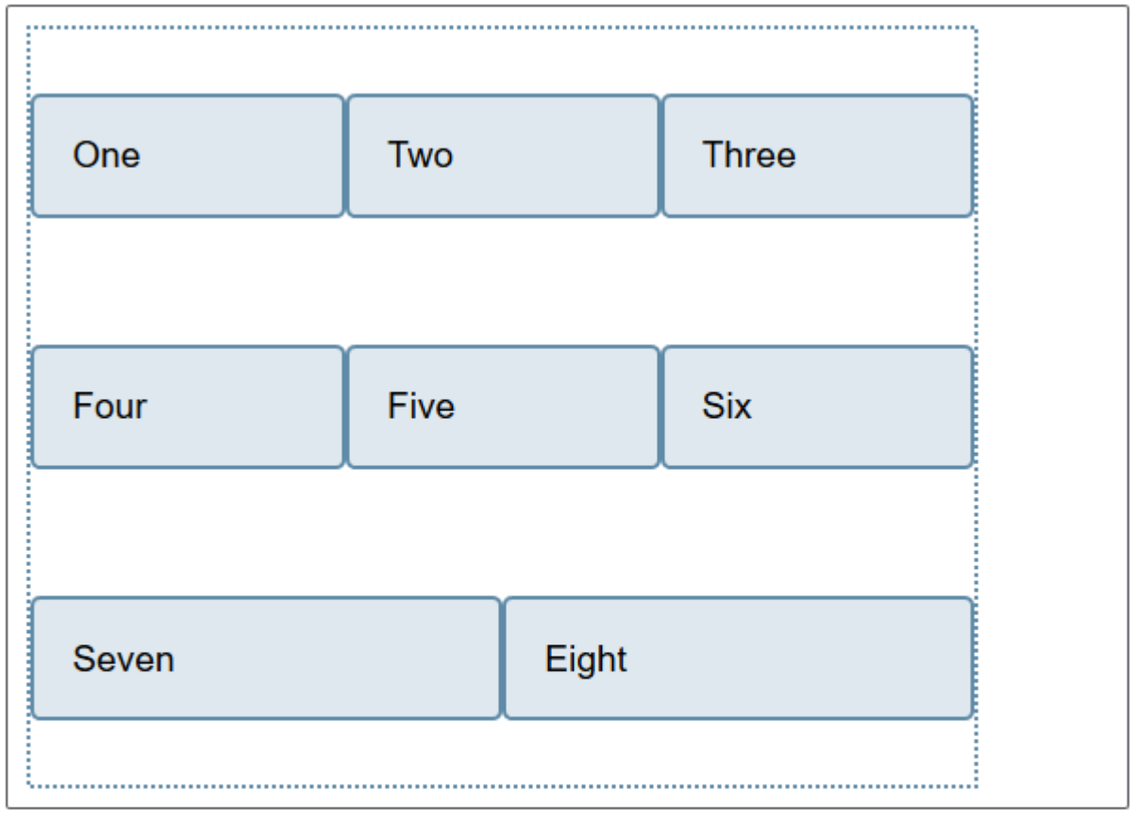



```
.box {
  display: flex;
  flex-wrap: wrap;
  height: 400px;
  align-content: center;
}
```

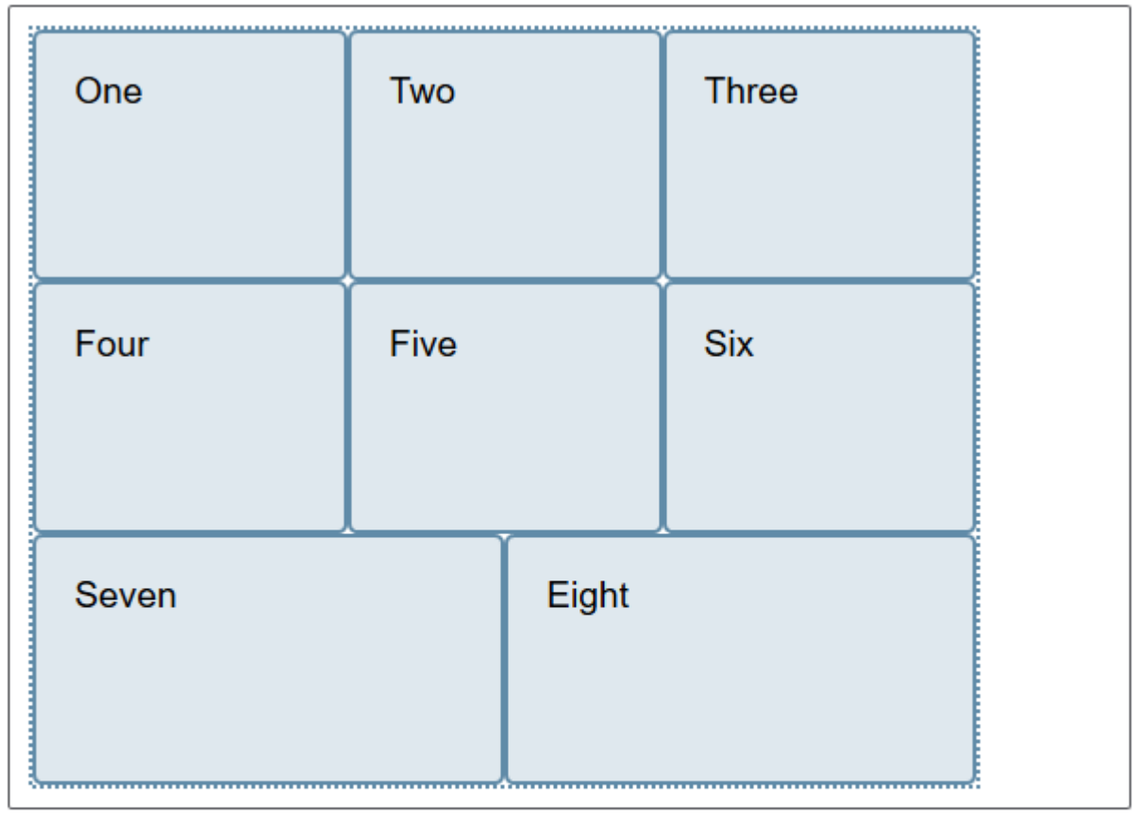



```
.box {
  display: flex;
  flex-wrap: wrap;
  height: 400px;
  align-content: space-between;
}
```

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout/Aligning_Items_in_a_Flex_Container




```
.box {
  display: flex;
  flex-wrap: wrap;
  height: 400px;
  align-content: space-around;
}
```




```
.box {
  display: flex;
  flex-wrap: wrap;
  height: 400px;
  align-content: stretch;
}
```

flex Property Example

`flex` is a shorthand for separate **`flex-grow`**, **`flex-shrink`**, and **`flex-basis`** properties.

The values 1 and 0 work like on/off switches.

```
li {
  flex: 1 0 200px;
}
```

In this example, list items in the flex container start at 200 pixels wide, are permitted to expand wider (`flex-grow: 1`), and are not permitted to shrink (`flex-shrink: 0`).

NOTE: The spec recommends always using the **`flex`** property and using individual properties only for overrides.

Skipping these three properties, see textbook or MDN for details

Changing Item Order

`order`

Values: *Number*

Specifies the order in which a particular item should appear in the flow (independent of the HTML source order):

- `order` is applied to the **flex item element**.
- The default is 0. Items with the same order value are placed according to their order in the source.
- Items with different order values are arranged from lowest to highest.
- The specific number value doesn't matter; only how it relates to other values (like z-index) matters.

Changing Item Order (cont'd)

Example:

"box3" has a higher order value (1) than the others with default order of 0. It appears last in the line even though it's third in the markup:

```
.box3 {
  order: 1;
}
```

Changing Item Order (cont'd)

Ordinal groups

Items that share the same order value are called an ordinal group.

Ordinal groups stick together and are arranged from lowest value to highest:

```
.box2, .box3 {
  order: 1;
}
```